Task 4: MongoDB Integration & SQL vs NoSQL Analysis

Overview

This task demonstrates NoSQL integration into the TechReads data pipeline. Task 4 covers data loading, querying, indexing strategies, and comparative performance analysis between MySQL and MongoDB.

Why MongoDB?

• **Flexible Schema:** Fields can vary between documents without ALTER TABLE operations.
• **Idempotent Upserts:** update_one(..., upsert=True) with book_url key prevents duplicates.
• **Scalability:** Horizontal scaling via sharding for distributed workloads.
• **Speed:** Eventual consistency allows faster writes compared to ACID MySQL transactions.

Data Loading Process

CSV data is loaded into MongoDB using PyMongo:
1. **Connection:** MongoClient('mongodb://localhost:27017/')
2. **Iteration:** Parse each CSV row
3. **Transformation:** Convert publication_year to int, price to float, rating to int
4. **Upsert:** collection.update_one({'book_url': url}, {'$set': doc}, upsert=True)
5. **Results:** Track upserted/updated document count

MongoDB Queries (Q1-Q4)

Q1: Rating Filter (rating >= 4)
collection.find({'rating': {'$gte': 4}}) — Identifies highly-rated books

Q2: Price Filter (price <= 20)
collection.find({'price': {'$lte': 20.0}}) — Finds affordable books

Q3: Sort by Price (DESC)
collection.find({}).sort('price', DESCENDING).limit(10) — Top 10 expensive

Q4: Year Filter (publication_year > 2018)
collection.find({'publication_year': {'$gt': 2018}}) — Recent Data Engineering titles (REQUIRED BY BRIEF)

Indexing Strategy (Learning Outcome 4)

Three strategic indexes were created to accelerate queries and demonstrate understanding of database optimization:
• **idx_rating (ASCENDING):** Speeds up Q1 filtering by rating
• **idx_price (DESCENDING):** Speeds up Q3 sorting and Q2 filtering by price
• **idx_year (ASCENDING):** Speeds up Q4 filtering by publication year

Impact: At small scale (20 books), timing differences are modest (milliseconds). At production scale (millions of documents), indexes reduce full-collection scans from O(n) to O(log n) lookups — a critical performance optimization.

Performance Comparison: MySQL vs MongoDB

The same four queries were executed on both databases BEFORE and AFTER creating indexes. Results demonstrate that proper indexing strategy, not database technology alone, determines query performance.

Aspect	MySQL (SQL/Relational)	MongoDB (NoSQL/Document)
Schema	Fixed structure; ALTER TABLE required	Flexible; add fields dynamically
Data Integrity	ACID transactions; foreign keys	Eventual consistency
Complex Joins	Multi-table joins optimized	Denormalization preferred
Scalability	Vertical (larger hardware)	Horizontal (sharding)
Write Speed	Slower (ACID overhead)	Faster (eventual consistency)
Field Changes	Costly (schema migration)	Free (add field to document)
Best For	Structured, relational data	Semi-structured, hierarchical data

Recommendations: Choose MySQL for strict ACID requirements and complex relational queries. Choose MongoDB for flexible schemas, rapid iteration, and horizontal scaling.

Key Implementation Details

1. MongoDB Connection & Collection Setup:

```
from pymongo import MongoClient
client = MongoClient('mongodb://localhost:27017/')
collection = client['techreads_db']['books']
```

2. Data Transformation & Upsert:

```
for _, row in df.iterrows():
  publication_year = int(year_raw) if valid else None
  price = float(str(row['price']).replace('GBP','').replace('£',''))
  doc = {'book_url': url, 'title': title, 'price': price, ...}
  collection.update_one({'book_url': url}, {'$set': doc}, upsert=True)
```

3. Index Creation:

```
collection.create_index([('rating', ASCENDING)], name='idx_rating')
collection.create_index([('price', DESCENDING)], name='idx_price')
collection.create_index([('publication_year', ASCENDING)], name='idx_year')
```

Key Learnings

1. **Schema Flexibility:** NoSQL eliminates schema migration overhead for evolving data models
2. **Idempotency:** Upserts on unique keys ensure pipelines can be safely re-run
3. **Index Strategy:** B-tree indexes are essential at scale; optimize based on query patterns
4. **Polyglot Persistence:** Modern pipelines often use multiple database technologies
5. **Trade-offs:** SQL provides integrity; NoSQL provides flexibility. Neither is universally superior.

Task 4 Completion Summary

Requirement	Status
MongoDB integration & data loading	✓ Complete
Q1-Q4 queries with results	✓ Complete
Three strategic indexes created	✓ Complete
Performance comparison (before/after)	✓ Complete
SQL vs NoSQL analysis with recommendations	✓ Complete
Publication year filter (brief requirement)	✓ Complete
Upsert-based deduplication	✓ Complete